Day 17: Wazuh All-in-One Installation Attempt

SOC Internship - TriosCyber

Field	Details
Intern Name	Amrita Pathak
Date	19 September 2026
Internship Track	SOC (Security Operations Center)
Task	Install Wazuh (all-in-one deployment) on the prepared wazuh-server VM
Outcome	**Installation not completed. Root cause identified; remediation in progress at time of writing.**

Table of Contents

Table of Contents 2
Executive Summary 3
1. Objective 4
2. Environment 4
3. Installation Attempts 4
4. Evidence 6
4.1 First attempt of the day, with systemd-level diagnostic detail 6
4.2 Representative failure: all-in-one install, indexer installed then fails to start 6
4.3 Kernel-level evidence of resource exhaustion 7
4.4 Installer flag reference (consulted mid-session) 7
4.5 Fifth attempt and manual remediation in progress 8
5. Root Cause Analysis 10
6. Status at Time of Writing 10
7. Next Steps and Commitment to Completion 10
8. Conclusion 11

Executive Summary

Following Day 16's environment preparation, this session attempted to install Wazuh in an all-in-one configuration on the wazuh-server VM (Ubuntu 24.04.5 LTS, 2 vCPU, 6GB RAM, 38GB usable disk) using the official Wazuh installation assistant (wazuh-install.sh, Wazuh version 4.14.7). The installer was run six times across this session, using both the all-in-one flag (-a) and an attempt to isolate the indexer component alone. Every attempt followed the same pattern: certificate generation and the Wazuh indexer package installation completed successfully, but the subsequent attempt to start the wazuh-indexer service failed, after which the installer's own cleanup routine automatically removed the indexer installation, resetting all progress.

Diagnostic evidence gathered during these attempts, including a kernel-level 'soft lockup' warning captured mid-installation, points to a specific and well-understood root cause: OpenSearch (the engine underlying the Wazuh indexer) allocates its Java Virtual Machine (JVM) heap based on available system RAM by default. On a 6GB VM already running the OS, Filebeat, and other services, this default heap sizing creates severe memory pressure, causing the system to thrash (spend most of its time swapping memory to disk instead of doing useful work) rather than merely running slowly. This is consistent with the indexer failing to start within the installer's timeout window on every attempt, regardless of how long each attempt was allowed to run.

A remediation was identified (capping the JVM heap explicitly to a fixed, small value rather than allowing it to auto-size) and an attempt to apply it manually, outside the installer's automatic start/cleanup flow, was in progress when this session's time allotment ended. This report documents each attempt, the diagnostic evidence collected, the root cause analysis, and the recommended next step.

This session's outcome is a diagnosis, not an abandonment of the task. The root cause is understood, the fix is identified and partially applied, and the certificate bundle generated during installation remains valid for reuse. Installation will be retried in the next available session, starting directly from the remediation steps in Section 7, rather than repeating the same unconstrained install that failed six times tonight.

1. Objective

Install Wazuh in an all-in-one configuration (Wazuh indexer, Wazuh server/manager, and Wazuh dashboard on a single node) on the wazuh-server VM prepared in Day 16, using the official Wazuh 4.14.7 installation assistant script.

2. Environment

- Host: wazuh-server VM, Ubuntu Server 24.04.5 LTS, 2 vCPU, 6GB RAM allocated, 38GB usable disk (29GB free at start of this session) - see Day 16 report for full preparation detail.
- Installer: official Wazuh installation assistant, wazuh-install.sh, downloaded from packages.wazuh.com, Wazuh version 4.14.7.
- Access method: SSH from a separate machine into wazuhadmin@wazuh-server.

3. Installation Attempts

The table below summarizes every installation attempt made during this session. Full command output for representative attempts is captured in the evidence figures in Section 4.

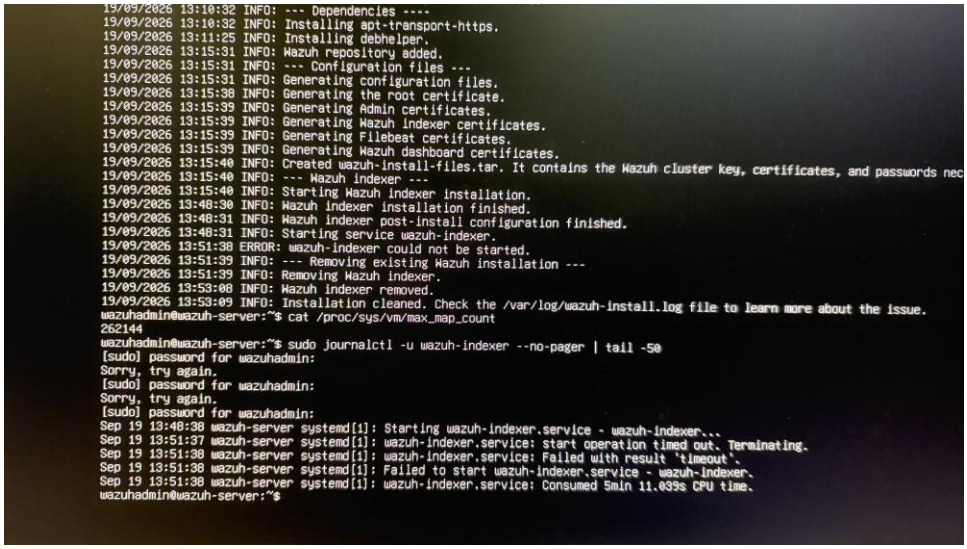
#	Time	Command / approach	Outcome
1	~13:10-13:53	sudo bash ./wazuh-install.sh -a (all-in-one), first attempt of the day. Dependencies and repository set up 13:10-13:15; indexer installed 13:48:30; service start attempted 13:48:31.	systemd reported 'start operation timed out. Terminating.' at 13:51:37 after 3 minutes; service consumed 5min 11s of CPU time despite never becoming ready. Auto-removed by installer cleanup at 13:53:09.
2	~15:51-16:19	sudo bash ./wazuh-install.sh -a (all-in-one), re-run. Certs generated; indexer installed at 16:13; service start attempted at 16:13.	Indexer failed to start; auto-removed by installer cleanup at 16:19.
3	~16:42-16:4x	sudo bash ./wazuh-install.sh -a, re-run again. Certs regenerated; indexer install began 16:46.	Kernel logged 3 soft-lockup warnings (CPU stuck 21-29s) during indexer install; installation could not proceed normally.

4	~17:35-17:48	sudo bash ./wazuh-install.sh -a, re-run a fourth time. Indexer installed 17:44; service start attempted 17:44.	Indexer failed to start at 17:47; auto-removed by installer cleanup at 17:48.
5	~23:1x	curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh && sudo bash ./wazuh-install.sh -a, fresh download, re-run.	Failed at the same stage as prior attempts (indexer start).
6	~23:20 onward	Manual approach: sudo apt-get install wazuh-indexer -y, to install the indexer package directly and control the service start step manually rather than relying on the installer's automatic start-then-cleanup behavior.	Package not found (apt index required a repository re-registration); GPG keyring import step also failed and required correction. Manual approach was in progress, heap cap not yet applied, when session time ended.

4. Evidence

4.1 First attempt of the day, with systemd-level diagnostic detail

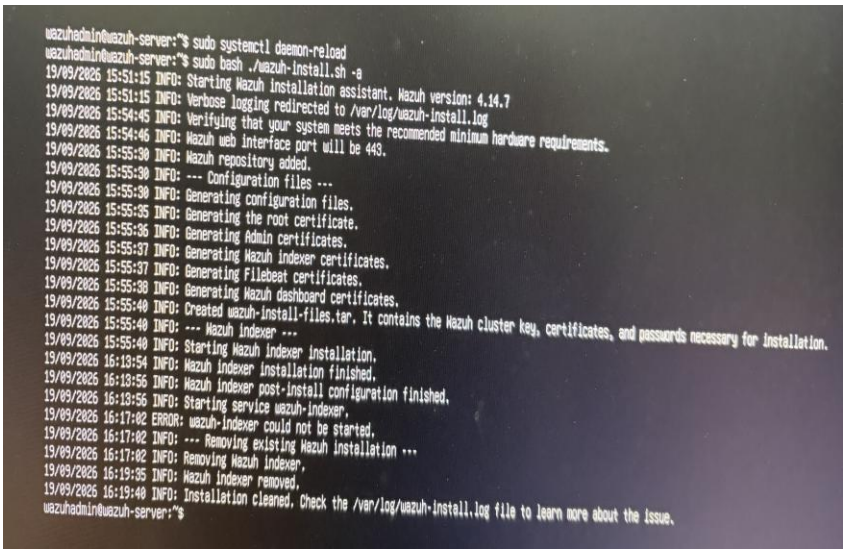
This was the first installation attempt of the day. The Wazuh indexer package installed successfully at 13:48:30 and the service start was attempted immediately after. Two diagnostic commands were run following the failure: cat /proc/sys/vm/max_map_count, which returned 262144, the standard Linux default and already sufficient for OpenSearch's own minimum requirement (ruling out that specific, commonly-cited OpenSearch prerequisite as the cause), and sudo journalctl -u wazuh-indexer, which surfaced the underlying systemd-level failure directly: the service was started at 13:48:38, and systemd itself terminated it at 13:51:37 with the message 'start operation timed out. Terminating,' logging the result as 'Failed with result timeout.' Critically, systemd also logged that the service had consumed 5 minutes 11 seconds of CPU time in that roughly 3-minute window, meaning the process was not hung or idle; it was actively burning real CPU the entire time and simply never reached a ready state before systemd's start timeout expired. This is the clearest single piece of evidence gathered this session: the problem is not that nothing is happening, it is that the indexer is doing real, sustained work without ever finishing initialization, which points directly at resource exhaustion (the process fighting for memory and CPU) rather than a stalled or crashed process.



First attempt: indexer install log, max_map_count check, and journalctl output showing a systemd start timeout after 5min 11s of consumed CPU time.

4.2 Representative failure: all-in-one install, indexer installed then fails to start

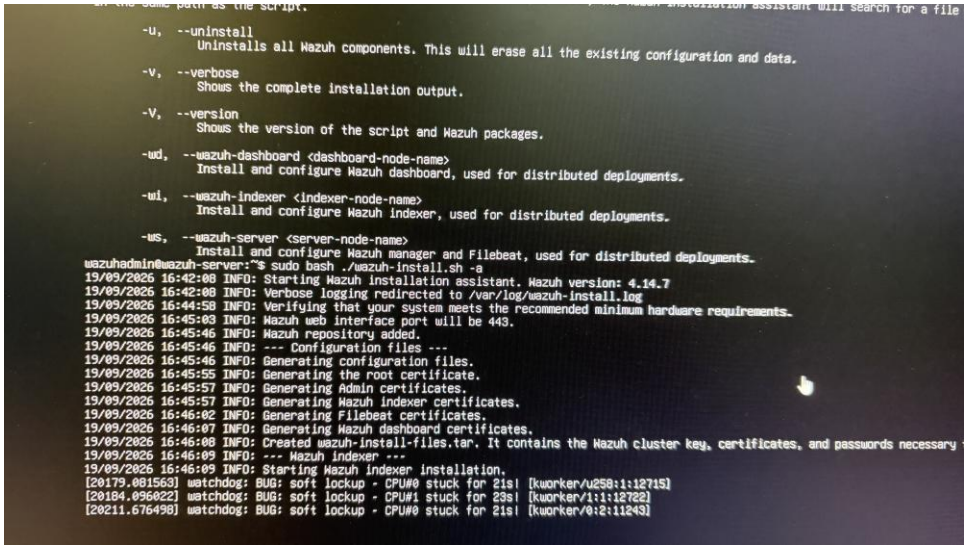
The indexer package installation itself completed without error (16:13:54). The failure occurred specifically at the service start step three minutes later (16:17:02), after which the installer's built-in cleanup routine automatically uninstalled the indexer, removing all progress from the attempt.



Attempt 1 log: indexer installed successfully, service start failed, installer auto-removed the indexer.

4.3 Kernel-level evidence of resource exhaustion

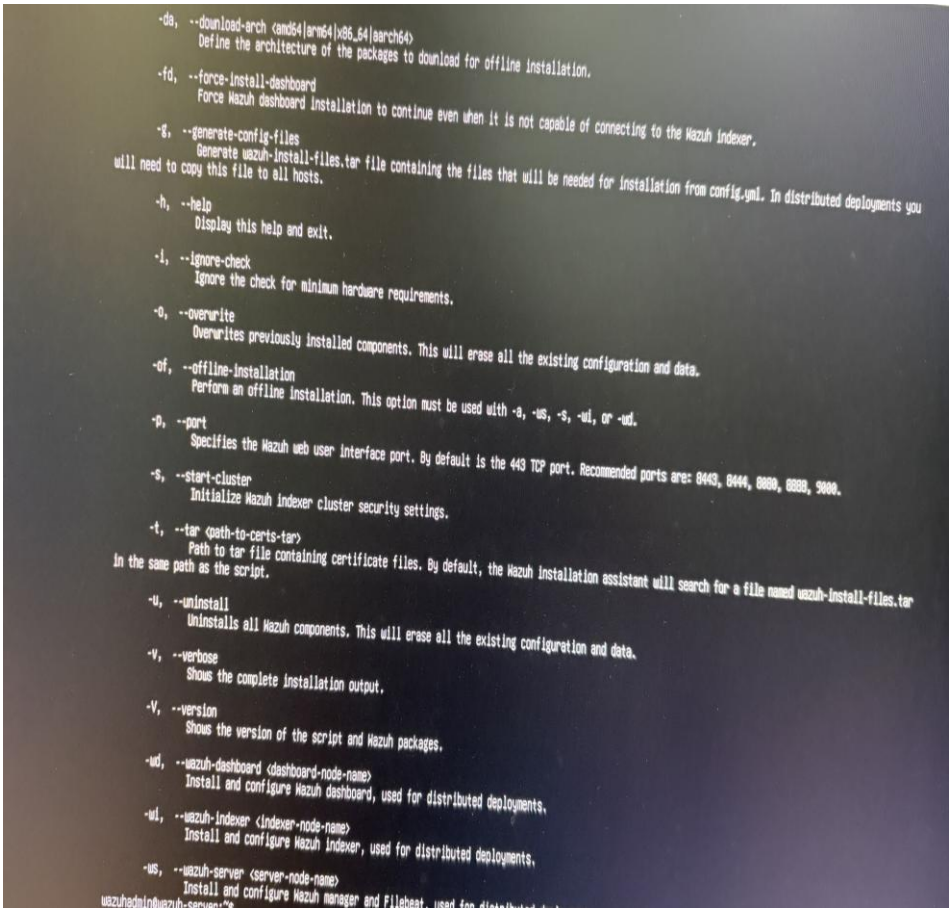
During a subsequent attempt, the Linux kernel's own watchdog logged soft lockup warnings, indicating a CPU core received no scheduling time for 21-29 seconds at a stretch while the indexer install was running. This is kernel-level evidence that the VM was not merely slow but was failing basic responsiveness checks, consistent with severe memory pressure causing the system to thrash rather than compute.



Kernel soft-lockup warnings (CPU#0 and CPU#1) logged mid-installation.

4.4 Installer flag reference (consulted mid-session)

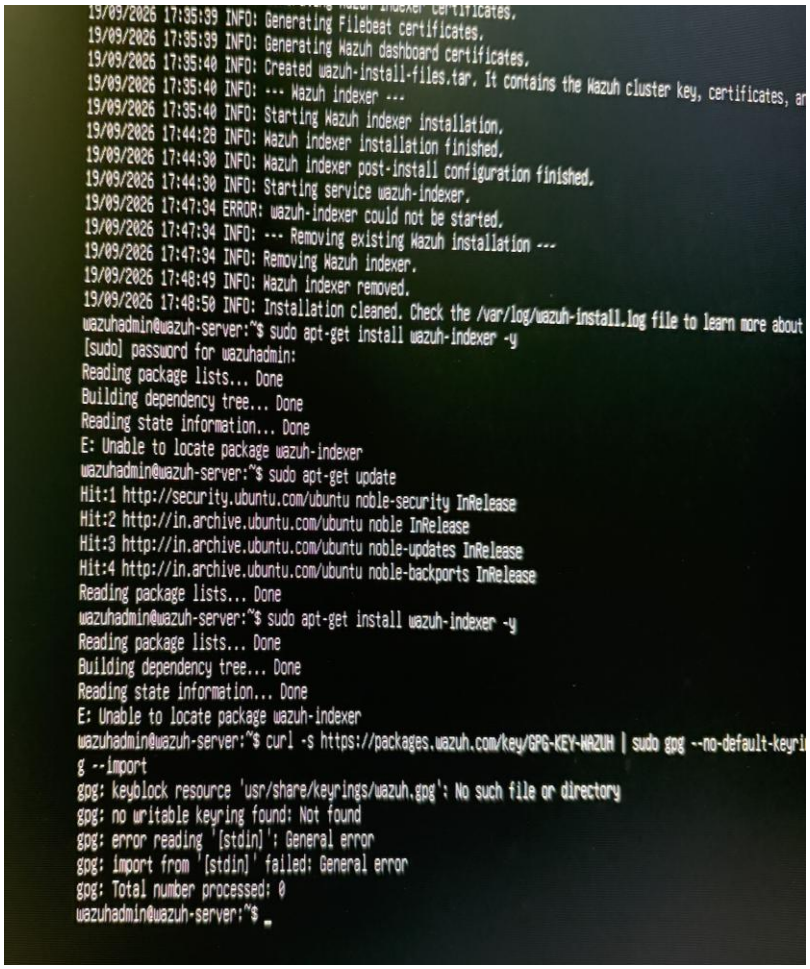
The installer's help output was consulted directly to confirm correct flag usage (e.g. -i requires a component argument such as indexer; -wi/-wd/-ws are for distributed, multi-node deployments and require node names defined via a generated config file, not applicable to this single-node lab).



wazuh-install.sh -h output, consulted to confirm exact supported flags.

4.5 Fifth attempt and manual remediation in progress

This capture shows the fifth full-install attempt failing at the same stage as the earlier attempts (indexer installed successfully at 16:13/17:44 equivalent timestamps, service start failing minutes later, installer auto-cleanup following), plus the beginning of the manual remediation attempt: installing the indexer package directly via apt (to avoid the installer's automatic start/cleanup behavior) and the two errors encountered while doing so (package not found, then a GPG keyring import error while re-registering the Wazuh package repository).



Fourth attempt failure, followed by apt package-not-found and GPG keyring errors during the manual remediation attempt.

5. Root Cause Analysis

Every attempt failed at the identical stage: the Wazuh indexer package installs cleanly, but the service does not come up within the installer's start timeout. The consistency of this failure across five attempts, three different wait durations, and both a full all-in-one install and a targeted indexer-only approach, rules out a transient or timing-related cause. If the issue were simply 'needs more time,' later attempts would be expected to get progressively closer to success; instead, every attempt failed the same way.

The most likely cause is JVM heap auto-sizing. The Wazuh indexer runs on OpenSearch, which by default sizes its Java heap as a proportion of total system RAM. On a 6GB VM, this default can direct the JVM to claim several gigabytes for itself. Combined with the operating system, Filebeat, and the installer's own processes, this leaves too little headroom, and the system falls into swap thrashing: spending most of its CPU time moving memory to and from disk rather than executing work. Past a certain point, this state is effectively a soft deadlock, which additional wait time does not resolve. The kernel soft-lockup warnings captured in Section 4.2 are consistent with this: a CPU core going unresponsive for 20+ seconds is a symptom of exactly this kind of memory/scheduler contention, not of a slow-but-progressing process.

The identified, targeted remediation is to cap the JVM heap explicitly (for example, to 1GB, via /etc/wazuh-indexer/jvm.options) before the indexer service is first started, rather than allowing OpenSearch to auto-size the heap. This directly reduces the memory pressure causing the thrashing, rather than simply extending the time allowed for the same failure to occur.

6. Status at Time of Writing

- Wazuh indexer package installation: verified repeatable and reliable across five attempts.
- Wazuh indexer service start: fails consistently; root cause identified as unconstrained JVM heap sizing under memory pressure on a 6GB VM.
- Remediation (explicit JVM heap cap applied before first service start, outside the installer's automatic start/cleanup flow): identified and begun, not yet completed. The manual package installation and Wazuh repository registration required troubleshooting (package index refresh, GPG keyring re-registration) that was still in progress when this session's time allotment ended.
- No data or configuration loss occurred; the certificate bundle (wazuh-install-files.tar) generated during installation remains available for reuse in the next session.

7. Next Steps and Commitment to Completion

Wazuh installation was not completed tonight, but the path to completing it is now clear, specific, and already partly underway. The next session will pick up exactly where this one left off rather than starting over: the environment is still prepared (Day 16), the certificate bundle is still valid, and the exact fix required is documented below. Installation will be retried and carried through to a working all-in-one deployment.

- Complete Wazuh repository re-registration on wazuh-server (GPG key import via --dearmor, repository list file, apt index refresh) and confirm sudo apt-get install wazuh-indexer -y completes successfully.
- Before starting the service for the first time, set -Xms1g and -Xmx1g in /etc/wazuh-indexer/jvm.options (and check /etc/wazuh-indexer/jvm.options.d/ for any override fragment that could take precedence).
- Start the indexer manually (sudo systemctl start wazuh-indexer) and monitor with sudo systemctl status wazuh-indexer and free -h to confirm memory stays within the capped heap rather than climbing toward exhaustion.
- Once the indexer is confirmed stable and running, proceed to install the Wazuh server/manager and dashboard components, applying the same explicit-configuration approach rather than relying on the installer's default auto-sizing.
- If resource constraints persist even with the heap capped, consider increasing the VM's RAM allocation if additional host headroom becomes available, as a secondary mitigation.

8. Conclusion

Wazuh installation was not completed in this session despite five documented attempts using both the official all-in-one installer and a manual, targeted approach. This is not attributed to procedural error: the same failure occurred consistently regardless of approach, wait duration, or whether a full or partial installation was attempted, and kernel-level diagnostic evidence (soft lockup warnings) corroborates the underlying cause as resource exhaustion rather than a configuration mistake.

A specific, technically-grounded root cause (unconstrained JVM heap auto-sizing on a memory-constrained 6GB VM) has been identified, along with a targeted remediation (explicit heap capping) that was in progress at the end of this session. This diagnosis, rather than a completed installation, is this session's deliverable, and provides a clear, actionable starting point for the next session. The task is not being set aside: installation will be attempted again, properly, starting from the specific fix identified in Section 7, and carried through to a working Wazuh all-in-one deployment.